

# Unit: Graph Optimization

---

## What is Graph Optimization?

Graph Optimization is the process of finding the best solution in a graph.

It helps solve problems like:

- Finding the shortest route in Google Maps
  - Connecting cities using minimum cable
  - Scheduling tasks in the correct order
  - Sending maximum data through a network
- 

## 1. Dijkstra's Algorithm

---

### Definition

Dijkstra's Algorithm finds the shortest distance from one source vertex to every other vertex.

**Condition:** All edge weights must be positive.

---

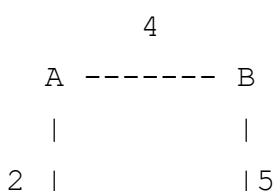
### Real-life Example

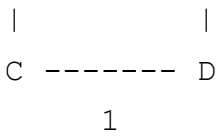
Suppose you are in Chennai and want to find the shortest route to all nearby cities.

Dijkstra checks all possible routes and always chooses the minimum distance.

---

### Example Graph





Find the shortest distance from **A**.

---

## Step 1

Distance (A) = 0

B =  $\infty$

C =  $\infty$

D =  $\infty$

Visited = {}

---

## Step 2

Choose the smallest distance vertex.

A = 0

Visit **A**.

Update neighbours.

B = 4

C = 2

---

## Step 3

Smallest unvisited vertex:

C = 2

Visit **C**.

Update **D**.

D = 3

Because

$$2 + 1 = 3$$

---

## Step 4

Visit **D**.

Current distance:

3

Update **B**.

$$3 + 5 = 8$$

Already

$$B = 4$$

No update.

---

## Step 5

Visit **B**.

Algorithm Ends.

## Final Distances

$$A = 0$$

$$B = 4$$

$$C = 2$$

$$D = 3$$

---

## Algorithm

1. Set source distance = 0.
2. Set all other distances =  $\infty$ .
3. Pick the minimum distance vertex.
4. Mark it as visited.
5. Update neighbouring vertices.
6. Repeat until all vertices are visited.

---

# Python Code

```
import heapq

graph = {
    'A': [('B', 4), ('C', 2)],
    'B': [('A', 4), ('D', 5)],
    'C': [('A', 2), ('D', 1)],
    'D': [('B', 5), ('C', 1)]
}

def dijkstra(graph, start):
    distance = {v: float('inf') for v in graph}
    distance[start] = 0

    pq = [(0, start)]

    while pq:
        dist, node = heapq.heappop(pq)

        for neighbour, weight in graph[node]:
            new_dist = dist + weight

            if new_dist < distance[neighbour]:
                distance[neighbour] = new_dist
                heapq.heappush(pq, (new_dist, neighbour))

    return distance

print(dijkstra(graph, 'A'))
```

## Output

```
{'A': 0, 'B': 4, 'C': 2, 'D': 3}
```

---

# Complexity

## Time Complexity

$$O((V + E) \log V)$$

### Space Complexity

$$O(V)$$

---

## Advantages

- Fast
  - Efficient
  - Best for positive edge weights
- 

## Disadvantages

- Cannot handle negative edge weights.
- 

## 2. Bellman-Ford Algorithm

---

### Definition

Bellman-Ford finds the shortest path even when the graph contains negative edge weights.

---

### Example

A → B (4)

A → C (2)

C → D (-3)

D → B (1)

---

### Step-by-Step

## Initial

$$A = 0$$

$$B = \infty$$

$$C = \infty$$

$$D = \infty$$

---

## Relax Edges

$$A \rightarrow B$$

$$B = 4$$

$$A \rightarrow C$$

$$C = 2$$

$$C \rightarrow D$$

$$2 + (-3) = -1$$

$$D \rightarrow B$$

$$-1 + 1 = 0$$

## Final

$$A = 0$$

$$B = 0$$

$$C = 2$$

$$D = -1$$

---

## Algorithm

Repeat **V - 1** times:

- Relax every edge.

Then perform one extra iteration.

If any distance changes,

**Negative Cycle Exists**

---

# Python Code

```
edges = [  
    ('A', 'B', 4),  
    ('A', 'C', 2),  
    ('C', 'D', -3),  
    ('D', 'B', 1)  
]  
  
vertices = ['A', 'B', 'C', 'D']  
  
distance = {v: float('inf') for v in vertices}  
distance['A'] = 0  
  
for i in range(len(vertices) - 1):  
    for u, v, w in edges:  
        if distance[u] + w < distance[v]:  
            distance[v] = distance[u] + w  
  
print(distance)
```

---

# Complexity

$O(VE)$

---

# Advantages

- Supports negative edge weights.
- 

# Disadvantages

- Slower than Dijkstra's Algorithm.
- 

# Dijkstra vs Bellman-Ford

---

| Dijkstra              | Bellman-Ford             |
|-----------------------|--------------------------|
| Positive weights only | Negative weights allowed |
| Faster                | Slower                   |
| Uses Heap             | Uses Edge Relaxation     |
| $O((V+E) \log V)$     | $O(VE)$                  |

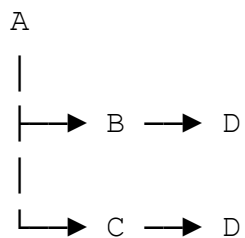
## 3. Topological Sorting

### Definition

Topological Sorting arranges the vertices of a **Directed Acyclic Graph (DAG)** so that every directed edge  $u \rightarrow v$  places **u before v**.

It is mainly used for **task scheduling**.

### Example



### Possible Answer

A  
B  
C  
D

### Applications

- Course scheduling
- Project planning



- Software dependency management
  - Build systems
- 

## Kahn's Algorithm Steps

1. Find indegree of every vertex.
  2. Insert all vertices with indegree 0 into a queue.
  3. Remove a vertex, add it to the result, and decrease the indegree of its neighbours.
  4. Repeat until the queue becomes empty.
- 

## Python Code

```
from collections import deque

graph = {
    0: [1, 2],
    1: [3],
    2: [3],
    3: []
}

indegree = [0] * 4

for u in graph:
    for v in graph[u]:
        indegree[v] += 1

q = deque()

for i in range(4):
    if indegree[i] == 0:
        q.append(i)

while q:
    node = q.popleft()
    print(node, end=" ")

    for nei in graph[node]:
        indegree[nei] -= 1
```

```
if indegree[nei] == 0:
    q.append(nei)
```

## Output

0 1 2 3

---

# 4. Minimum Spanning Tree (MST)

---

## Definition

A **Minimum Spanning Tree (MST)** is a subset of edges that:

- Connects all vertices.
  - Has no cycles.
  - Has the minimum total edge weight.
- 

## Example

```
      1
A ----- B
|  \      |
|   \2    |
4    \     |
|     \    |
C-----
```

Choose

A-B = 1

B-C = 2

Total Cost

3

---

## Prim's Algorithm

## Idea

Start with any vertex and repeatedly add the smallest edge that connects a visited vertex to an unvisited vertex.

## Complexity

$O(E \log V)$

---

# Kruskal's Algorithm

## Idea

Sort all edges by weight and repeatedly add the smallest edge that does not form a cycle.

## Complexity

$O(E \log E)$

---

## Prim vs Kruskal

| Prim                    | Kruskal                  |
|-------------------------|--------------------------|
| Starts from a vertex    | Starts from sorted edges |
| Uses Priority Queue     | Uses Union-Find          |
| Better for dense graphs | Better for sparse graphs |

---

# 5. Introduction to Network Flow

---

## Definition

A **Flow Network** is a directed graph where each edge has a capacity.

The objective is to determine the **maximum possible flow** from a **Source** to a **Sink**.

---

# Example

Source

|



A

|



B

|



Sink

Each edge has a capacity.

Example

Source  $\rightarrow$  A = 10

A  $\rightarrow$  B = 8

B  $\rightarrow$  Sink = 6

Maximum Flow

6

Because the last edge limits the total flow.

---

## Applications

- Water distribution
- Internet routing
- Traffic management
- Airline scheduling
- Supply chain optimization

---

## Summary

---

| Algorithm        | Purpose                             | Complexity                    |
|------------------|-------------------------------------|-------------------------------|
| Dijkstra         | Shortest path (positive weights)    | $O((V+E) \log V)$             |
| Bellman-Ford     | Shortest path with negative weights | $O(VE)$                       |
| Topological Sort | Task scheduling in a DAG            | $O(V+E)$                      |
| Prim             | Minimum Spanning Tree               | $O(E \log V)$                 |
| Kruskal          | Minimum Spanning Tree               | $O(E \log E)$                 |
| Network Flow     | Maximum flow from source to sink    | Depends on the algorithm used |

---